UNIVERSITY OF TRIESTE

Department of Mathematics, Informatics and Geosciences

Master's Degree in Data Science and Artificial Intelligence

Curriculum: Foundations of Artificial Intelligence and Machine Learning

Unsupervised State Bucketing for Mixture of Experts in Resource-Constrained Chess Engines

Graduation session: 23 October 2026

Candidate
Omar Cusma Fait
Student ID SM3800018

Supervisor
Prof. Alessio Ansuini

Internship supervisor
Prof. Marco Zennaro
ICTP

Academic Year 2025/2026

Lorem ipsum
dolor sit amet.
— Cicero – *De finibus bonorum et malorum* —

Ad maiora!

Abstract

In the context of board games, a common approach to increasing model performance is to partition the state space and allocate distinct experts to each region. While this is the central idea behind Mixture of Experts (MoE) architectures, the bucketing is almost always defined manually, requiring domain-specific expertise, and yielding potentially suboptimal partitions.

While unsupervised bucketing using hidden layer activations has been explored, this approach may not yield partitions that *maximise expert specialisation*. Activations capture what the model *knows*, not what it *needs* to adjust. In this work, we ask whether we can use the model's learning dynamics — the sample-gradients — to create a partition that is more effective. Our hypothesis is that clustering gradients (which encode the direction of desired weight updates) will produce buckets that are inherently better suited for expert fine-tuning, leading to more specialised experts.

We propose a novel unsupervised bucketing method based on sample-gradients — the gradients of the loss with respect to the head parameters of a base model. These vectors encode the direction in weight space that would improve the model's prediction for each individual data point. We apply the method to chess, using a standard NNUE (Efficiently Updatable Neural Network) as the base model trained on 5 million positions labelled with outcome probabilities (WDL) by a strong value function (Lc0).

For each position, we compute the normalised sample-gradient with respect to the head parameters, and apply a clustering algorithm to define the buckets. We then train a lightweight linear dispatcher to predict the bucket from the L1 activations, enabling fast, deterministic routing at inference time. Finally, we fine-tune the expert heads on each bucket, starting from the base model, while keeping the L1 weights frozen. This yields a set of fast, specialised experts, each adapted to a distinct region of the state space, without requiring handcrafted bucketing.

Contents

Chapter 1

Introduction

1.1 Motivation

A chess engine spends most of its time asking the same question: *how good is this position?* On a desktop, that question can be answered by a large network; on a microcontroller, it cannot. The target hardware for this work is the *Wio Terminal* (192 KB RAM, 500 KB flash), where the evaluation function must be small, integer-friendly, and cheap to run at every node of an alpha-beta search. *Efficiently Updatable Neural Networks* (**NNUE**) are the current standard: an *accumulator layer* (sparse first layer that tracks the board incrementally), followed by a small fully connected head.

With that architecture, a single head will struggle to capture the nuances of openings, quiet middlegames, tactical melees, and simplified endgames. Different positions may require understanding of very diverse aspects of the game. Mixture-of-Experts (**MoE**) evaluation is the natural solution: partition the state space into buckets, and assign an *expert head* to each region, while keeping a shared representation. In chess engines, this partition is almost always written by hand — piece count, king location, game phase. Those rules are cheap and interpretable, but they are game-specific heuristics and are not necessarily optimal. They encode what a programmer thinks is a distinct regime, not what the model actually needs in order to specialise.

Unsupervised alternatives exist. Clustering hidden-layer activations, for example, groups positions that look similar in the eyes of the network. The weakness of this approach lies in the fact that similarity of representation is not the same as similarity of *learning signal*: activations describe what the model already encodes, not the direction in which its head should change. If the goal is expert specialisation, the partition should be defined by that direction. This thesis is motivated by the idea that a lightweight, data-driven bucketing — cheap enough for an on-device NNUE, and not tied to chess-specific features — is both a practical need for tiny engines and a more general way to mix experts over a state space.

1.2 Problem Statement

The central problem addressed in this thesis is the design of an efficient, *data-driven* method for partitioning the state space of a chess engine into regions of specialist expertise, within the tight *inference-time* resource constraints of embedded devices.

More specifically, consider a standard NNUE evaluation function composed of a frozen shared representation W_{L1} and a trainable head (W_{L2}, W_{out}). Given a dataset of positions $\mathcal{D} = \{(s_i, v_i)\}$, we can train a base model w_{base}. To improve upon this base model via expert specialization, we seek a partition of the state space into B buckets $\{\mathcal{D}_1, \ldots, \mathcal{D}_B\}$ such that training a separate head on each bucket yields a set of *task vectors* $\delta_i = \theta_i - \theta_{\text{base}}$ that are maximally diverse.

This objective is complicated by the fact that the partition must be discovered from the training data, yet the criterion for "good" specialization—diversity of task vectors—lives in the space of model weights, not in the input space. At the same time, the resulting routing mechanism must be *lightweight* enough to run on a microcontroller at every node of an alpha-beta search, ruling out expensive computations such as evaluating multiple full networks. Current heuristic bucketing strategies (e.g., based on piece count or king position) are computationally cheap but encode arbitrary *game-specific* assumptions about which positions are similar, failing to capture the actual learning signal that drives head specialization.

This work, therefore, investigates whether it is possible to learn an unsupervised partition directly from instance-specific gradients—using them as a proxy for the direction in which each head should specialize—and whether such a partition can be deployed via a lightweight dispatcher that respects the memory and computational constraints of an embedded device, without sacrificing the quality of the resulting mixture-of-experts evaluation.

1.3 Contributions

Recent work has explored the idea of clustering samples by their gradient directions to create specialized experts. **ELREA** [1] partitions training instructions by gradient directions to reduce optimization conflicts, while **GradientSpace** [2] clusters LoRA gradients and deploys a lightweight encoder-based router for fast single-expert inference. Both demonstrate that gradient-space partitioning is a powerful principle for mixture-of-experts architectures.

However, these methods are designed for **large language models** (LLMs), where the parameter space is enormous, experts are implemented as LoRA adapters, and inference latency is measured in milliseconds rather than microseconds. The problem of state-space partitioning for a chess evaluation function on an embedded device presents a fundamentally different set of constraints: the model must fit in a very small amount of memory, routing must occur at every node of an alpha-beta search with negligible overhead, and the *state space* partition should be discoverable from the data by looking at the model only.

Building on these prior works, this thesis introduces a perspective on unsupervised gradient-based bucketing adapted to the domain of chess engines. We propose a Mixture-of-Experts NNUE architecture where the state space is partitioned by clustering per-sample gradients of the loss with respect to the head parameters (W_{L2}, W_{out}), capturing the learning signal that drives expert specialization without relying on handcrafted heuristics such as piece count or game phase. At inference time, a lightweight dispatcher—trained to predict bucket assignments from frozen L1 activations—routes each position to a single specialized head with negligible overhead, avoiding the cost of *ensembling* or *on-the-fly gradient computation*. We validate our approach empirically on chess, comparing against single-head baselines and heuristic bucketing strategies, and demonstrate that *gradient-informed partitioning* yields more distinct experts and improved evaluation quality on a dataset of positions labeled with WDL values from a strong engine.

Chapter 2

Background and Related Work

2.1 Chess Engines and Evaluation Functions

At the core of every chess engine lies an **evaluation function** $f : \mathcal{S} \to \mathbb{R}$ that assigns a scalar value to a board position, indicating the expected outcome from that position (typically from the perspective of the player to move). This value guides the search algorithm—usually a variant of **alpha-beta search** with iterative deepening—by pruning unpromising branches and selecting the most promising moves.

2.1.1 Classical Evaluation

For decades, chess evaluation functions were handcrafted. A classical evaluator is a weighted linear combination of chess-specific features:

$$f(s) = \sum_k w_k \cdot \phi_k(s)$$

where $\phi_k(s)$ are feature functions and w_k are scalar weights. Typical *chess-specific* features include material balance, piece-square tables (PSTs), mobility, king safety, pawn structure. These features were carefully tuned by chess experts over decades, using a combination of intuition, empirical testing, and later, automated optimization techniques.

While classical evaluators are extremely fast, they suffer from fundamental limitations. The human-designed features encode the human intuition about chess, which may not align with the optimal understanding of the game. Moreover, these models do not take into account the positions as a whole; the same parameters give a positional bonus for a central knight whether the position is a quiet middlegame or a tactical melee, even though the knight's practical value may differ dramatically across phases.

2.1.2 The Shift to Neural Evaluation

The limitations of handcrafted features led to the adoption of neural networks as evaluation functions. **AlphaZero** [3] demonstrated that a deep convolutional network, trained via self-play reinforcement learning, could surpass the best classical engines. However, these networks are computationally expensive—requiring millions of operations per evaluation—making them unsuitable for resource-constrained devices or for engines that must evaluate millions of positions per second.

2.1.3 NNUE: A Hybrid Approach

The **Efficiently Updatable Neural Network (NNUE)** architecture, first introduced in the *shogi* engine *YaneuraOu* and later adopted by *Stockfish*, strikes a pragmatic balance. NNUE combines the representational power of a neural network with the incremental efficiency of classical evaluators.

The architecture consists of two ingredients: a sparse first layer called the *accumulator*, and a small fully-connected head.

The accumulator layer maps a binary representation of the board to a hidden representation $h \in \mathbb{R}^d$ via $h = W_{L1} \cdot x$, where x is a sparse binary vector (typically 768 or 1024 dimensions) indicating the presence of pieces on squares. The key insight is that a chess move changes only a few bits of x, so h can be **updated incrementally** by adding and subtracting the corresponding columns of W_{L1}, rather than by recomputing the entire matrix-vector product from scratch. This makes the computation of L1 essentially *free*.

The second ingredient, the fully connected head, is typically one or two hidden layers followed by a scalar output, mapping h to the position value. In our case, we found easier to train a probability distribution across all the three possible game result for the player; win, draw, and loss (WDL), by minimizing the cross-entropy between the output of the teacher and the student.

A complete NNUE engine also leverages **alpha-beta search** with iterative deepening: starting from the root position, the engine searches deeper and deeper, using the NNUE evaluation at leaf nodes to guide the pruning and ordering of moves. At every node of this search tree, the evaluation function is called hundreds or thousands of times, making its speed critical. Also, being trained on enormous amounts of data, a NNUE doesn't suffer from the *horizon problem* as much as a PST does.

2.1.4 Value targets: centipawns, EV, WDL

Conceptual comparison after NNUE / AlphaZero: scalar centipawns (classical / Stockfish-style NNUE), expected value $v \in [-1, 1]$ (AlphaZero), full WDL distribution. This is a representation choice, not only a metric. Loss choice (soft CE on WDL vs MSE on cp/EV) is fixed in Sections 3.2–3.3 and 4.2.

2.1.5 Why Evaluation Must Be Cheap

The search tree explored by an **alpha-beta engine** grows exponentially with depth. Even with effective move ordering and pruning techniques, the number of evaluated positions increases dramatically as the search goes deeper. A chess engine that searches deeper consistently outperforms one that searches shallower, as each additional ply reveals tactical patterns and strategic nuances that would otherwise remain hidden.

This means that any overhead added to the evaluation function is directly subtracted from the engine's effective search depth. If the evaluator becomes slower, the engine must reduce its search depth to maintain the same response time—sacrificing playing strength in the process.

On an embedded device such as the Wio Terminal, this constraint is even more severe. Memory is limited, floating-point operations are expensive, and every instruction counts. Any routing mechanism for a mixture-of-experts NNUE must add only a trivial cost—ideally, a handful of integer operations or a simple table look-up—to avoid degrading the engine's search performance.

In short, the evaluation function must be expressive enough to assess positions accurately, yet cheap enough to be called millions of times during a game. This trade-off is the central engineering challenge addressed by this thesis.

2.1.6 Current State of the Art for Tiny Hardware: Cfish

For resource-constrained devices, the most relevant reference implementation is **Cfish**, a port of the Stockfish chess engine written in plain C by Ronald de Man. While Stockfish is written in C++ and targets desktop-class hardware, Cfish strips away the C++ abstractions and compiles to a much smaller binary, making it viable for microcontrollers and other embedded platforms. The engine can be compiled with various evaluation backends, including a pure NNUE mode that excludes the classical handcrafted evaluation entirely, resulting in an even smaller executable.

From an architectural standpoint, Cfish shares the same core search and evaluation logic as Stockfish. The input to the evaluation function is a board position represented internally as a compact bitboard structure. The NNUE evaluation itself follows the *halfkp* feature set: for each piece on the board, the network considers the piece's square together with the square of the king of the same color, producing a set of activated features that are fed into the accumulator. The accumulator is updated incrementally as moves are made, avoiding recomputation from scratch. The output of the NNUE is a scalar value, typically in centipawns, representing the expected advantage from the perspective of the side to move.

Cfish is particularly relevant to this thesis for two reasons. First, it demonstrates that a full-featured NNUE engine *can* be made to run on constrained hardware, provided the implementation is careful about memory layout and avoids C++ overhead. Second, it serves as a practical baseline: any Mixture-of-Experts NNUE architecture proposed for tiny devices should be at least as fast and compact as Cfish in its pure NNUE mode, while offering improved evaluation accuracy through expert specialization. In this sense, Cfish represents both the *state of the art* and the *performance target* for the embedded chess engine developed in this work.

2.2 Mixture of Experts

Multiple expert networks over sub-domains. Learned vs. fixed routing. Uses in vision, NLP, and RL, and what carries over to a tiny chess eval.

LoRA: how ELREA / GradientSpace implement experts on LLMs. Contrast LoRA adapters vs a tiny NNUE head. Not used in this method.

2.3 Sample Gradients

Per-example gradients w.r.t. head parameters as a representation of the learning signal. Why they differ from activations. Pointers to gradient-clustering literature.

2.4 Bucketing in NNUE

Handcrafted buckets: piece count, king location, piece presence (queen, bishop pair, . . .). Why they are cheap, and why they may not match what the model needs.

2.5 Related Work

Efficiency (pruning, quantization). Distillation / teacher–student. Self-play RL (AlphaZero, Lc0). Work closest to sample-gradient clustering and MoE routing.

Chapter 3

Method: Unsupervised Bucketing via Sample Gradients

3.1 Overview

Five-step pipeline: train a base model; compute sample-gradients; cluster them; train a dispatcher; fine-tune expert heads.

3.2 Notation and Definitions

We introduce here the formal notation used throughout this chapter and the remainder of the thesis. The notation is organized into sets, scalars, vectors and matrices, functions, and key operators.

3.2.1 Sets

Symbol	**Description**
$\mathcal{S}$	The space of all possible chess positions (states).
$\mathcal{D}$	The training dataset of positions with labels: $\mathcal{D} = \{(s_i, v_i)\}_{i=1}^{N}$.
$\mathcal{P} = \{\mathcal{D}_1, \ldots, \mathcal{D}_B\}$	A partition of the state space into B buckets, where each $\mathcal{D}_i \subset \mathcal{S}$ is a non-empty subset.
Θ	The space of all model parameters (weights).
$\mathbb{R}$	The set of real numbers.

3.2.2 Scalars

Symbol	Description
B	The number of experts (buckets).
N	The total number of positions in the training dataset.
N_i	The number of positions in bucket $\mathcal{D}_i$.
h	The hidden dimension of the NNUE accumulator.
d_{in}	The input dimension of the NNUE accumulator (844 in this work).
$v_i \in \mathbb{R}$	The scalar label (expected reward) for position s_i.
η	The learning rate used for gradient updates.

3.2.3 Vectors and Matrices

Symbol	Type	Description
$W_{L1} \in \mathbb{R}^{d_{\text{in}} \times h}$	Matrix	Weights of the first layer (accumulator), **frozen** after base training.
$W_{L2} \in \mathbb{R}^{h \times h}$	Matrix	Weights of the second layer.
$W_{\text{out}} \in \mathbb{R}^{h \times 3}$	Matrix	Weights of the output layer (3 logits for WDL).
$w_{\text{base}} \in \mathbb{R}^{P}$	Vector	All parameters of the base model: $w_{\text{base}} = \text{vec}(W_{L1}, W_{L2}^{\text{base}}, W_{\text{out}}^{\text{base}})$.
$\theta_i \in \mathbb{R}^{P}$	Vector	All parameters of the expert model i: $\theta_i = \text{vec}(W_{L1}, W_{L2}^{(i)}, W_{\text{out}}^{(i)})$.
$\delta_i \in \mathbb{R}^{P_{\text{head}}}$	Vector	The **task vector** for expert i: $\delta_i = \theta_i^{\text{head}} - \theta_{\text{base}}^{\text{head}}$, where $\theta^{\text{head}} = \text{vec}(W_{L2}, W_{\text{out}})$.
$\Delta_i \in \mathbb{R}^{P_{\text{head}}}$	Vector	The **per-sample gradient** for position s_i: $\Delta_i = \nabla_{w_{\text{head}}} \mathcal{L}(\hat{f}_{w_{\text{base}}}(s_i), v_i)$.
$x_i \in \{0,1\}^{d_{\text{in}}}$	Vector	The sparse binary feature vector for position s_i.
$h_i = W_{L1} \cdot x_i \in \mathbb{R}^{h}$	Vector	The accumulator output (L1 activation) for position s_i.

3.2.4 Functions and Models

Symbol	Description
$f_w : \mathcal{S} \to \mathbb{R}^3$	The NNUE evaluation function parameterized by weights w, producing WDL logits.
$\hat{f}_w : \mathcal{S} \to \mathbb{R}$	The NNUE scalar value function: $\hat{f}_w(s) = \text{softmax}(f_w(s))_W - \text{softmax}(f_w(s))_L$.
$g_\phi : \mathcal{S} \to \{1, \ldots, B\}$	The **dispatcher** function, parameterized by ϕ, mapping a position to a bucket index.
$\mathcal{L}(y, v)$	The loss function, typically soft cross-entropy on WDL probabilities.
$\mathcal{L}_{\text{acc}}(y, v)$	The accumulated loss over a batch or epoch.

3.2.5 Key Operators

Symbol	Description
$\mathrm{vec}(\cdot)$	The vectorization operator, flattening a matrix into a column vector.
$\nabla_w \mathcal{L}$	The gradient of the loss with respect to the parameters w.
$\lVert\cdot\rVert$	The Euclidean (L2) norm.
$\mathrm{softmax}(z)_k = \frac{e^{z_k}}{\sum_j e^{z_j}}$	The softmax function over logits z.
$d_{\cos}(u, v) = 1 - \frac{u \cdot v}{\lVert u\rVert \lVert v\rVert}$	The cosine distance between two vectors u and v.

3.2.6 Relationship Between Δ_i and δ_i

A crucial distinction in this work is between **per-sample gradients** Δ_i and **task vectors** δ_i:

- $\Delta_i = \nabla_{w_{\text{head}}} \mathcal{L}(\hat{f}_{w_{\text{base}}}(s_i), v_i)$ is the gradient of the loss with respect to the head parameters, evaluated at a **single position** s_i using the base model. It represents the direction in which the head would need to move to reduce the loss on that specific position.
- $\delta_i = \theta_i - \theta_{\text{base}}$ is the actual change in the head parameters after **fine-tuning** on all positions in bucket $\mathcal{D}_i$. It is the accumulated effect of the per-sample gradients over the bucket.

The central hypothesis of this work is that clustering positions by their Δ_i yields a partition $\mathcal{P}$ for which the resulting δ_i are maximally diverse, meaning each expert specializes in a distinct region of the state space.

3.3 Step 1: Train the Base Model

Shared dual-POV L1, L2, output head. Soft cross-entropy on WDL labels from a strong teacher (Lc0). This is the chosen loss (not MSE on centipawns or EV).

3.4 Step 2: Compute Sample Gradients

Per-sample gradients w.r.t. head parameters. Normalisation (L2 or standardisation). Storage and compute cost.

3.5 Step 3: Cluster Sample Gradients

K-Means and alternatives (DBSCAN, Density Peak; some choose B). How to pick B. Validation: inertia, silhouette, interpretability.

3.6 Step 4: Train the Dispatcher

Input: L1 activations h. Output: bucket id b. Linear layer or small MLP. Cross-entropy. Needed because gradients are not available at inference.

3.7 Step 5: Fine-Tune Expert Heads

Freeze L1; fine-tune one head per bucket from the base model. Result: B specialised experts.

3.8 Generalisation Beyond Chess

The method needs only a state space, a model, a loss, and a target. Other games, robotics, world-model settings.

Chapter 4

Implementation: NNUE and MoE for Chess

4.1 Dataset and Teacher

The quality of an NNUE evaluation function depends critically on the dataset used for training. For this work, we construct a dataset of approximately **5 million chess positions** extracted from human games and labeled with high-quality value estimates from a strong teacher network, **Leela Chess Zero** (Lc0), the *spiritual* successor of *AlphaZero*.

4.1.1 Dataset Construction

The raw positions are sourced from **Lichess** monthly PGN archives, containing standard-rated games played by humans across all time controls. We filter games to include only those with at least 16 moves, excluding very short games that often end in early blunders or resignations and would introduce noisy or uninformative positions into the training set. From each remaining game, we sample positions randomly (e.g., one every 20), ensuring a diverse and representative collection of states across all phases of play, and mostly avoiding highly correlated positions.

The dataset retains the natural distribution of game phases found in human play: a majority of middlegame positions, with fewer openings and endgames. We intentionally avoid resampling to balance phases, as the natural distribution better reflects the positions the engine will encounter during actual play. Each position is stored as a FEN string along with the multiplicity of the position—a visit count that may be used for optional weighting during training.

A small fraction of the dataset is also a collection of interesting tactical positions, and high level bot games. They capture a portion of the state space that may be outside of regular human play.

4.1.2 Feature Encoding

For the NNUE model, each FEN string is encoded as a **sparse binary feature vector** of length 844. This encoding is designed to capture both the positional and tactical structure of the board in a form suitable for the accumulator layer.

The 844 features are divided into two categories:

- **716 base features**: These encode piece-square pairs, representing the presence of each piece type on each square. The feature set is pruned to remove impossible pawn ranks and compressed to reduce redundancy (e.g., the king plane is stored in a compact form). In the **844-dim SARDINE encoder**, the king plane is **compressed** from 64 squares to **32**, saving features.
- **128 tactical features**: These encode dynamic aspects of the position, specifically which pieces are under attack and which pieces are attacking the king. These features provide the network with explicit information about immediate tactical threats.

A key design choice is the **dual-POV** encoding: for each position, the encoder produces two sets of sparse indices—one from the perspective of the **side-to-move** (STM) and one from the **opponent's** perspective (obtained by flipping the board rank-wise and swapping colors). This dual representation allows the network to learn symmetric evaluations and is consistent with the NNUE architecture's ability to evaluate positions from either player's viewpoint.

The encoded features are pre-computed and stored in `.npz` slices for efficient loading during training.

4.1.3 Data Splits

The dataset is partitioned into training and test splits. The **training set** consists of approximately 5 million positions, distributed across 165 slices for balanced I/O and stochastic sampling. The **test set** comprises a random portion of 5% of the position that are held out of the training set.

4.1.4 Teacher Model: Lc0

To provide accurate target labels, we use **Lc0** (Leela Chess Zero) as the teacher model. Lc0 is a *convolutional neural network* trained via self-play reinforcement learning, following the AlphaZero paradigm. Its value head outputs a probability distribution over the three possible game outcomes—Win, Draw, Loss—from the perspective of the side-to-move:

$$p_{\mathrm{WDL}}(s) = \mathrm{softmax}(\mathrm{logits}(s)) = (p_W, p_D, p_L)$$

From this distribution, we compute the scalar expected reward:

$$v(s) = p_W - p_L \in [-1, +1]$$

which represents the expected outcome of the game from the current position. This scalar is the training target for the NNUE value head.

Lc0 is chosen as the teacher for several reasons. First, it natively outputs WDL probabilities, which align directly with the NNUE's output head. Second, its strength—rated well above 3500 Elo—makes it a highly reliable source of positional evaluations. Finally, Lc0 is open-source and provides pre-trained networks, making the labelling pipeline reproducible.

For this work, we label positions using Lc0's **latest best network** (e.g., `791556.pb.gz` from the Lc0 training server). We run Lc0 in UCI mode with `-show-wdl` enabled and evaluate each position with a single MCTS search. While depth-1 evaluations may occasionally miss short-term tactics, the resulting label noise is acceptable given the target Elo range of the engine (approximately 1700). For a cleaner but more expensive relabelling, one could increase the search depth.

4.1.5 Labelling Pipeline

The complete labelling pipeline consists of four steps. First, we parse Lichess PGNs and sample positions uniformly from each game, saving FEN strings and visit counts. Then, for each unique FEN, we invoke Lc0 in UCI mode to obtain WDL probabilities from the STM perspective. We then procede to save the WDL probabilities and the scalar expected reward alongside the FEN and visit counts in JSON format. Finally, in the encoding step we pre-compute the 844-dimensional sparse feature vectors (both STM and opponent POVs) and store them in `.npz` slices for efficient training.

4.2 Base NNUE Architecture

The base NNUE (Efficiently Updatable Neural Network) model serves as the foundation upon which the Mixture of Experts extension is built. Its architecture is designed to balance representational capacity with the stringent memory and computational constraints of the target hardware. The model follows the dual-perspective paradigm introduced by the original NNUE design, but incorporates modifications tailored to the specific feature set and bucketing objectives of this work.

4.2.1 Model Overview

The base model f_θ is a feed-forward neural network with three parameterised layers: a shared sparse first layer (L1), a dense second layer (L2), and a linear output head, with *softmax* activations. The input to the model is the sparse binary feature representation described in Section 4.1.2, consisting of 844 active features per perspective. The model processes both the side-to-move (STM) and opponent perspectives through the same L1 layer, producing two accumulator vectors that are later concatenated and passed through the remaining layers.

Formally, the model computes:

$$h_{\text{own}} = W_{\text{L1}}\, x_{\text{own}}, \qquad h_{\text{opp}} = W_{\text{L1}}\, x_{\text{opp}},$$

where $x_{\text{own}}, x_{\text{opp}} \in \{0,1\}^{844}$ are the sparse feature vectors for the two perspectives, and $W_{\text{L1}} \in \mathbb{R}^{844 \times W}$ is the shared weight matrix of the accumulator layer. The output of the L1 layer is a pair of vectors $h_{\text{own}}, h_{\text{opp}} \in \mathbb{R}^{W}$, where W is the hidden dimension of the accumulator, set to $W = 64$ in this work.

4.2.2 Shared L1 Accumulator

The L1 layer, often referred to as the accumulator, is the defining component of the NNUE architecture. Its weight matrix W_{L1} is shared between the two perspectives, enabling the network to learn a common representation of board structure while retaining perspective-specific information through the different input features. The sparsity of the input features allows the accumulator to be updated efficiently: rather than recomputing the entire matrix-vector product for each new position, the network maintains the accumulator vector incrementally, adding or subtracting the contributions of features that change as pieces move.

The L1 activations are passed through a **CReLU** (Clipped Rectified Linear Unit) activation function, which maps each activation to the range $[0, 127]$:

$$a_{\text{own}} = \text{clamp}(h_{\text{own}}, 0, 127), \qquad a_{\text{opp}} = \text{clamp}(h_{\text{opp}}, 0, 127).$$

This clipping is essential for integer quantization, as it bounds the dynamic range of the accumulator values and allows the use of low-precision integer arithmetic during inference.

4.2.3 Side-to-Move Reorder

Before concatenating the two accumulator vectors, we apply a **side-to-move (STM) reorder** to ensure that the expert head always receives the perspective of the current player first. The reordering is governed by the binary flag stm_white $\in \{0, 1\}$, which indicates whether White is to move:

$$h_{\text{first}} = \begin{cases} a_{\text{own}}, & \text{if stm\_white} = 1, \\ a_{\text{opp}}, & \text{otherwise}, \end{cases}$$

$$h_{\text{second}} = \begin{cases} a_{\text{opp}}, & \text{if stm\_white} = 1, \\ a_{\text{own}}, & \text{otherwise}. \end{cases}$$

The two vectors are then concatenated to form the input to the L2 layer:

$$h = [h_{\text{first}} \parallel h_{\text{second}}] \in \mathbb{R}^{2W}.$$

This reordering step is critical for making the evaluation perspective-invariant: the network always receives the board from the point of view of the side to move, and the output $v \in [-1, +1]$ is always interpreted as the expected reward for the current player, regardless of colour.

4.2.4 L2 Layer and Output Head

The concatenated vector h is passed through a dense L2 layer with hidden dimension $H = 128$:

$$z = \text{ReLU}(W_{\text{L2}}\, h + b_{\text{L2}}),$$

where $W_{\text{L2}} \in \mathbb{R}^{2W \times H}$ and $b_{\text{L2}} \in \mathbb{R}^{H}$ are the weight matrix and bias of the L2 layer. The ReLU activation introduces non-linearity and has been shown to work well with the sparse accumulator features.

Finally, the L2 activations are projected to a **three-dimensional output** representing the logits for Win, Draw, and Loss probabilities:

$$\text{logits} = W_{\text{out}}\, z + b_{\text{out}},$$

where $W_{\text{out}} \in \mathbb{R}^{H \times 3}$ and $b_{\text{out}} \in \mathbb{R}^{3}$. During training, these logits are converted to probabilities via the softmax function:

$$p_{\text{WDL}}(s) = \text{softmax}(\text{logits}(s)) = (p_W, p_D, p_L).$$

The scalar evaluation used for search is obtained as $v = p_W - p_L$, the expected reward from the side-to-move perspective.

4.2.5 Training Objective

The model is trained to minimise the **soft cross-entropy** loss between the predicted WDL probabilities and the teacher labels provided by Lc0. For a batch of positions with targets $y_i = (\hat{P}_i(W), \hat{P}_i(D), \hat{P}_i(L))$ and model outputs $p_i = (P_i(W), P_i(D), P_i(L))$, the loss is:

$$\mathcal{L} = -\frac{1}{N} \sum_{i=1}^{N} \Big[\hat{P}_i(W) \log P_i(W) + \hat{P}_i(D) \log P_i(D) + \hat{P}_i(L) \log P_i(L)\Big].$$

This loss is well-suited for the task because the teacher labels are probability distributions, not point estimates. Using soft cross-entropy encourages the model to *match the full outcome distribution* rather than merely the scalar expected reward, providing a richer training signal. Additionally, the loss naturally handles the non-linearity of the scalar evaluation via the softmax, and its gradient does not saturate at extreme values, unlike the squared error on v.

4.2.6 Parameter Count and Model Size

With $W = 64$ and $H = 128$, the total number of trainable parameters is approximately 70,979. This compact size is deliberately chosen to fit within the memory constraints of the target hardware: the L1 weights ($844 \times 64 = 54{,}016$ int8 values) dominate the parameter count, while the L2 and output layers contribute only a small fraction. The model is therefore both computationally efficient and storage-friendly, with a footprint that can be further reduced through pruning and quantisation.

4.2.7 Training Protocol

The base model is trained on the full training set (approximately 5 million positions) using the Adam optimiser with a learning rate of 10^{-2}, linearly decayed to 10^{-3} over the course of training. We use a batch size of 1024 and train for up to 1000 epochs. The model's performance is evaluated on a held-out test set of random positions, 5% of the total dataset, ensuring that generalisation is measured on unseen data. The training is conducted on a *DGX Nvidia Spark GPU*.

4.3 Sample Gradient Computation

Implementation: `torch.autograd.grad`, per-sample grads w.r.t. the head. Storage (`.npy` / `.parquet`).

4.4 Clustering

Mini-Batch K-Means; Density Peak; other algorithms. Working B (e.g. 8, Stockfish-style — to check). t-SNE / PCA plots.

4.5 Dispatcher Training

Input [own‖opp] *vs. own side only. Linear $2W \to B$ (or $W \to B$). CE, Adam, short training.*

4.6 Expert Fine-Tuning

One sweep per bucket vs. longer training. Per-bucket test CE as the stopping signal.

4.7 Final MoE Model

Shared L1 + dispatcher + expert heads. Inference: L1 $\rightarrow$ dispatcher $\rightarrow$ selected expert $\rightarrow$ output.

4.8 Cfish as the host engine

Instantiate Section 2.1.6: what Cfish is (C port of Stockfish), `evaluate()`, search tree (α-β, iterative deepening, transposition table), Wio constraints (RAM, flash, nps).

4.9 Integration with Cfish

Hook `evaluate()` with the MoE NNUE. Quantization (int8 weights, int16 accumulators, int32 MAC). Memory: sparse L1, dispatcher and heads in flash.

Chapter 5

Experiments and Results

5.1 Experimental Setup

Splits (train / test sizes). Training on CPU, inference on Wio Terminal. Baselines: single-head NNUE; dense FFNN; handcrafted bucketing (piece count + queen presence).

5.2 Evaluation Metrics

Centipawn-based probabilistic Elo estimator. Test CE on WDL; MAE on expected value (even if the model is trained with CE, not MSE on EV); dispatcher accuracy; per-bucket CE vs. the base head.

Elo protocol (not just the name): matches, time control, opponent, BayesElo or SPRT, ACPL. Playing-strength numbers go in Section 5.4.

5.3 Results

We will list the CE of the models, and the Elo estimates.

5.3.1 Linear Model

5.3.2 Single-layer FFNN

5.3.3 Double-layer FFNN

5.3.4 Base NNUE

5.3.5 MoE NNUE — bucketing by L1 — fixed B

5.3.6 MoE NNUE — bucketing by L1 — variable B

5.3.7 MoE NNUE — bucketing with sample-gradients — fixed B

5.3.8 MoE NNUE — bucketing with sample-gradients — variable B

5.4 Results: Comparison of the Models

ACPL / Elo vs. baselines, using the protocol of Section 5.2. Cfish on Wio: nps, depth, move time.

5.5 Visualization: Clustering

t-SNE / PCA of sample-gradients. Dispatcher decision boundaries. Expert activation / routing histograms.

Exploratory analysis of the clusters — what type of position do they group together?

Chapter 6

Discussion

6.1 Interpretation of Results

What the numbers imply for specialisation, eval quality, and on-device cost. Whether clusters are semantically meaningful.

6.2 Limitations

Clustering quality, dispatcher errors, Wio memory/compute, sensitivity to B, dependence on Lc0 and on a single game.

6.3 Generalisation

Transfer to other domains. What must change (features, loss, target, hardware).

6.4 Comparison with Related Work

Vs. handcrafted NNUE bucketing. Vs. learned routing and gradient-clustering MoE (activations vs. sample-gradients; LLM LoRA vs. NNUE heads).

Chapter 7

Conclusion and Future Work

7.1 Summary of Contributions

Recap unsupervised sample-gradient bucketing, the lightweight MoE NNUE, and the chess-engine validation.

7.2 Future Work

Better clustering / automatic B; stronger dispatcher; other games or world-model settings; tighter on-device integration and Elo evaluation.

7.3 Closing Remarks

Short takeaway: data-driven partitions can replace handcrafted buckets when evaluation must stay tiny.

Appendix A

Supplementary Material

A.1 Extra tables and hyperparameters

Feature-encoder details, full hyperparameter lists, dataset statistics.

A.2 Extra plots

Clustering diagnostics, additional t-SNE / PCA, routing histograms that would interrupt Chapter 5.

A.3 Implementation notes

Bits that do not belong in the main implementation chapter (scripts, file formats, Cfish hook details).

Bibliography

[1] Yinghao Li et al. "Ensembles of Low-Rank Expert Adapters". In: *International Conference on Learning Representations*. 2025. arXiv: 2502.00089.

[2] Shrihari Sridharan et al. "GradientSpace: Unsupervised Data Clustering for Improved Instruction Tuning". In: (2025). arXiv: 2512.06678.

[3] David Silver et al. "A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play". In: *Science* 362.6419 (2018), pp. 1140–1144.

Ei fu. Siccome immobile,
Dato il mortal sospiro,
Stette la spoglia immemore
Orba di tanto spiro

Alessandro Manzoni – *Il Cinque Maggio*